



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

**NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA**



**ACCREDITED GRADE 'A'
BY NAAC**



**PERFECT SCORE OF 150/150 AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS**

1



Applied Machine Learning

Unit 05 – Lecture 10: Logistic Regression

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

The logit link, the Bernoulli likelihood, and the gradient $X^T(p - y)$

Topic focus: **a linear model for the log-odds, not for the label**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.; Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.; Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.

Line fails
oooo

Odds
oooo

Likelihood
oooooooooooo

Coefficients
ooooo

Threshold
oo

Multiclass
ooo

Separation
oooo

Summary
oo

Quick Links

Why a line fails

Odds and the logit

The likelihood

Odds ratios

The threshold

More classes

Separation

Summary

Agenda

Why linear regression fails for classification

Odds, log-odds and the sigmoid

The likelihood, the log-likelihood and the gradient

Reading the coefficients

The decision threshold is a choice

More than two classes

Perfect separation

Summary

How this lecture works

Every idea appears twice

1. **By hand** — eight tumours, a table of odds, three points and one gradient step. Small enough to check on paper, and the arithmetic is the examinable part.
2. **In code** — the same numbers out of `numpy` and `scikit-learn`, so you can see the library is doing your arithmetic and nothing else.

Commit before you look

Four slides ask you to predict a number before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

This is a **single-topic** lecture. We go all the way down: from “a straight line cannot be a probability” to $\nabla \mathcal{L} = X^T(p - y)$, **derived, not quoted**.

Where we are

What Unit V has given you so far

A linear model $\hat{y} = w^\top x + b$, least squares with a **closed form** $(X^\top X)^{-1} X^\top y$, maximum likelihood as the principle underneath it, and R^2 as the report.

What changes today

The target is no longer a quantity. It is a **yes or no**: malignant or benign, default or repay, spam or not.

Four questions

Why can a straight line not do this job?
What *can* a line model? What do we maximise instead of minimising squares?
And what does a coefficient mean afterwards?

One thing carries over unchanged

Maximum likelihood. L9 derived least squares *from* it, by assuming Gaussian noise. Today we assume **Bernoulli** outcomes instead, turn the same handle, and get something else.

One seed, and almost nothing to seed

The data

Bundled `load_breast_cancer` and `load_iris`, plus a few small hand-written tables. Nothing is downloaded and **nothing is randomly generated**.

The randomness

Exactly one source: the stratified train/test split, at `random_state=0`. The listing shows it.

And notice what is *not* random

Logistic regression is fitted by a deterministic solver on a **concave** objective — no initialisation to seed, no restart lottery, one optimum. We will prove that concavity rather than assert it. Every number in this deck reproduces exactly.

The target is a label, and a label is not a number

A probability lives in $[0, 1]$. A straight line does not.

Code the label 0 and 1 and fit least squares

Nothing stops you. `LinearRegression` will accept $y \in \{0, 1\}$ without a murmur and return a line. The trouble starts when you read the predictions.

Three separate failures

1. Predictions leave $[0, 1]$, so they are not probabilities.
2. A point far from the boundary **drags the boundary**, because squared error punishes being “too right”.
3. With three classes, coding them 1, 2, 3 asserts that class 2 sits between the other two.

ISLP §4.2 makes exactly this argument before introducing logistic regression. **Everything today follows from failure (1).**

By hand: eight tumours through a straight line

Size in mm; 0 = benign, 1 = malignant. $\bar{x} = 4.5$, $\bar{y} = 0.5$, $S_{xy} = 8$, $S_{xx} = 42$.

$$\hat{b}_1 = \frac{8}{42} = 0.190476, \quad \hat{b}_0 = 0.5 - 0.190476 \times 4.5 = -0.357143.$$

x	1	2	3	4	5	6	7	8
y	0	0	0	0	1	1	1	1
$\hat{y}$	− 0.1667	0.0238	0.2143	0.4048	0.5952	0.7857	0.9762	+ 1.1667

Two of the eight are not probabilities

A −16.7% chance of cancer, and a 116.7% chance. You can clip them, but then the model is no longer the model you fitted, and the clipping has no likelihood behind it.

Predict first #1

Now add one more patient: a **real, large, malignant** tumour at $x = 30$ mm. It is not an error. It agrees with every other malignant case.

Where does the least-squares $\hat{y} = 0.5$ boundary move to?

Predict first #1

Now add one more patient: a **real, large, malignant** tumour at $x = 30$ mm. It is not an error. It agrees with every other malignant case.

Where does the least-squares $\hat{y} = 0.5$ boundary move to?

	8 points	+ 1 distant point	shift
least squares boundary	4.5000	5.5517	+1.0517
logistic boundary	4.5000	4.5000	+0.000000
least squares slope	0.1905	0.0312	
misclassified by the refitted least-squares rule : 1 of 9			
misclassified by the refitted logistic rule : 0 of 9			
the point least squares now gets wrong: $x = [5.]$			

Predict first #1

Now add one more patient: a **real, large, malignant** tumour at $x = 30$ mm. It is not an error. It agrees with every other malignant case.

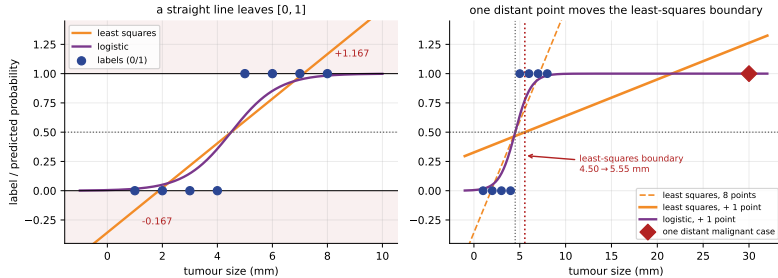
Where does the least-squares $\hat{y} = 0.5$ boundary move to?

	8 points	+ 1 distant point	shift
least squares boundary	4.5000	5.5517	+1.0517
logistic boundary	4.5000	4.5000	+0.000000
least squares slope	0.1905	0.0312	
misclassified by the refitted least-squares rule : 1 of 9			
misclassified by the refitted logistic rule : 0 of 9			
the point least squares now gets wrong: $x = [5.]$			

One correct point, and least squares loses a patient

Squared error is unhappy that $\hat{y}(30) = 1.29$ overshoots 1, so it flattens the whole line to reduce that one residual — and the boundary slides past a malignant tumour at 5 mm. **Logistic regression does not move at all**, because once $p \approx 1$ it has nothing left to gain.

Drawn: what a line does to a label



Left: the shaded bands are the forbidden region, and the orange line enters both. Right: the dashed line is least squares on eight points, the solid orange line is the same fit with one distant case added. **The purple curve is unmoved.**

Fix the range first, and everything else follows

Quantity	Range	A line?
probability p	$(0, 1)$	no
odds $p/(1 - p)$	$(0, \infty)$	no
log-odds $\ln \frac{p}{1-p}$	$(-\infty, \infty)$	yes

The two moves

1. $p \mapsto p/(1 - p)$ stretches $(0, 1)$ onto $(0, \infty)$: the upper bound is gone.
2. $\ln(\cdot)$ stretches $(0, \infty)$ onto $(-\infty, \infty)$: now *both* bounds are gone.

The whole idea, in one line

$$\ln \frac{p}{1 - p} = w^T x + b$$

The composition has a name

$\text{logit}(p) = \ln \frac{p}{1 - p}$, the **link function**: it links the bounded thing we want to the unbounded thing a linear model can make. Odds are the bookmaker's version of a probability — $p = 0.8$ is odds of 4.

We keep the linear model and change **what it is a model of**. All that remains is (a)

By hand: the odds table you should be able to write out

p	odds = $p/(1-p)$	log-odds = $\ln(\text{odds})$
0.01	0.0101	-4.5951
0.10	0.1111	-2.1972
0.20	0.2500	-1.3863
0.25	0.3333	-1.0986
0.50	1.0000	+0.0000
0.75	3.0000	+1.0986
0.80	4.0000	+1.3863
0.90	9.0000	+2.1972
0.99	99.0000	+4.5951

By hand: the odds table you should be able to write out

p	odds = $p/(1-p)$	log-odds = $\ln(\text{odds})$
0.01	0.0101	-4.5951
0.10	0.1111	-2.1972
0.20	0.2500	-1.3863
0.25	0.3333	-1.0986
0.50	1.0000	+0.0000
0.75	3.0000	+1.0986
0.80	4.0000	+1.3863
0.90	9.0000	+2.1972
0.99	99.0000	+4.5951

Read it twice. $p = 0.5$ is **log-odds** 0 — that is where the decision boundary will be. And the table is **antisymmetric**: $\text{logit}(0.2) = -\text{logit}(0.8)$, $\text{logit}(0.1) = -\text{logit}(0.9)$. Swapping the two classes flips the sign of everything.

Invert it, and the sigmoid appears

Start from the model and solve for p . Write $z = w^\top x + b$.

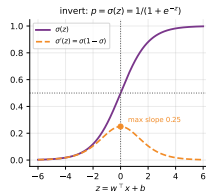
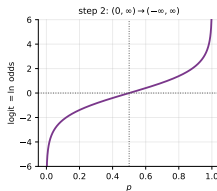
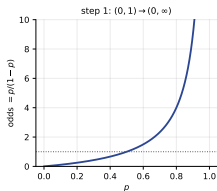
$$\ln \frac{p}{1-p} = z \implies \frac{p}{1-p} = e^z \implies p = e^z - p e^z \implies p(1 + e^z) = e^z$$

$$p = \sigma(z) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}}$$

Three facts to keep

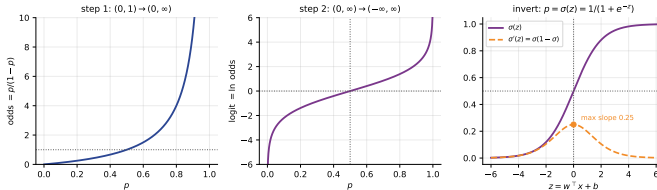
$\sigma(0) = \frac{1}{2}$; $\sigma(-z) = 1 - \sigma(z)$; $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. **The sigmoid was not invented. It is what the logit link becomes when you turn it round.**

Drawn, and checked in code



```
logit(sigma(z)) == z ? True    max error 1.11e-15
sigma(-z) == 1 - sigma(z) ? True
numerical derivative          0.01766 0.10499 0.19661 0.25000 0.19661 ...
sigma(z) * (1 - sigma(z))    0.01766 0.10499 0.19661 0.25000 0.19661 ...
max difference 6.28e-11
```

Drawn, and checked in code



```
logit(sigma(z)) == z ? True      max error 1.11e-15
sigma(-z) == 1 - sigma(z) ? True
numerical derivative           0.01766 0.10499 0.19661 0.25000 0.19661 ...
sigma(z) * (1 - sigma(z))      0.01766 0.10499 0.19661 0.25000 0.19661 ...
max difference 6.28e-11
```

The derivative identity is checked against a finite difference, not asserted. **It peaks at 0.25 where $z = 0$** , and that single number reappears twice today: in the gradient, and in the “divide by four” rule.

What are we maximising?

Each row is a single coin flip whose bias the model supplies:

$$y_i \mid x_i \sim \text{Bernoulli}(p_i), \quad p_i = \sigma(w^\top x_i + b).$$

The Bernoulli trick

$$P(y_i \mid x_i) = p_i^{y_i} (1 - p_i)^{1-y_i}$$

One formula for both cases. Put $y_i = 1$: the second factor is $(1 - p_i)^0 = 1$, leaving p_i . Put $y_i = 0$: the first factor is $p_i^0 = 1$, leaving $1 - p_i$. **The exponents are switches.**

Assuming the rows are independent given X , the likelihood is the product:

$$L(w, b) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}.$$

Take logs — and meet the cross-entropy loss again

A product of n numbers below 1 underflows; a sum does not. And $\ln$ is increasing, so the maximiser is unchanged.

$$\ell(w, b) = \ln L = \sum_{i=1}^n \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right]$$

You have already met this function

Negate it and divide by n and you have the **binary cross-entropy** of L2, which `sklearn` calls `log_loss`. Maximising the likelihood *is* minimising the cross-entropy. They are the same object seen from two directions — statistics and information theory.

So: $\mathcal{L}(w, b) = -\ell(w, b)$ is the loss we minimise, and the optimum is the maximum-likelihood estimate.

The gradient, one factor at a time

Differentiate ℓ with respect to a single weight w_j . Chain rule through $p_i = \sigma(z_i)$ and $z_i = w^\top x_i + b$.

$$\frac{\partial \ell}{\partial w_j} = \sum_i \underbrace{\left[\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i} \right]}_{\partial \ell / \partial p_i} \cdot \underbrace{p_i(1 - p_i)}_{\partial p_i / \partial z_i} \cdot \underbrace{x_{ij}}_{\partial z_i / \partial w_j}$$

Multiply the first two brackets out — this is the step worth doing on paper:

$$\left[\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i} \right] p_i(1 - p_i) = y_i(1 - p_i) - (1 - y_i)p_i = y_i - y_i p_i - p_i + y_i p_i = y_i - p_i.$$

The $p_i(1 - p_i)$ cancels exactly. That cancellation is the reason logistic regression is workable at all.

The result: prediction minus truth

$$\frac{\partial \ell}{\partial w_j} = \sum_i (y_i - p_i) x_{ij}, \quad \nabla_w \mathcal{L} = X^\top (p - y)$$

How to read it

The gradient is the design matrix transposed, times **the error vector**. Column j of the gradient is “how much feature j correlates with the mistakes I am currently making”. The intercept column of ones gives $\sum_i (p_i - y_i)$.

Where you have seen this before

Least squares has gradient $X^\top (Xw - y)$ — *also* $X^\top (\text{prediction} - \text{truth})$. Same shape, different prediction. That is not a coincidence: both are generalised linear models with their canonical link.

By hand: three points, one gradient step

X carries an intercept column. Start at $w = (b, w_1) = (0, 0)$, so every $z_i = 0$ and every $p_i = \sigma(0) = 0.5$.

$$X = \begin{pmatrix} 1 & -2 \\ 1 & 0 \\ 1 & 2 \end{pmatrix}, \quad y = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \quad p = \begin{pmatrix} 0.5 \\ 0.5 \\ 0.5 \end{pmatrix}, \quad p - y = \begin{pmatrix} +0.5 \\ +0.5 \\ -0.5 \end{pmatrix}.$$

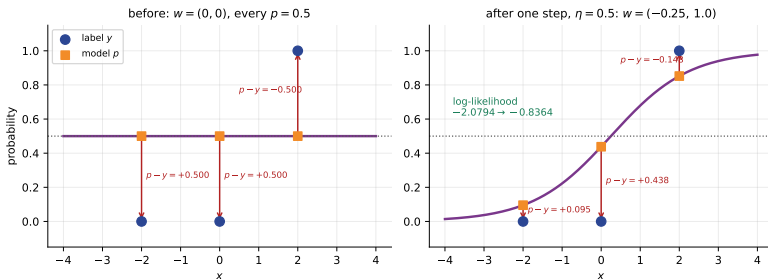
$$X^T(p - y) = \begin{pmatrix} 0.5 + 0.5 - 0.5 \\ (-2)(0.5) + (0)(0.5) + (2)(-0.5) \end{pmatrix} = \begin{pmatrix} +0.5 \\ -2.0 \end{pmatrix}.$$

With $\eta = 0.5$: $w \leftarrow (0, 0) - 0.5(0.5, -2.0) = (-\mathbf{0.25}, +\mathbf{1.0})$.

Then check that it helped

New $z = (-2.25, -0.25, 1.75)$, new $p = (0.0953, 0.4378, 0.8520)$, and the log-likelihood moves $-2.0794 \rightarrow -0.8364$. **All three probabilities moved towards their labels.**

Drawn: the same step



Blue circles are the labels, orange squares the model, and the red arrow on each point is $p - y$: exactly the number that enters the gradient. **Left, every arrow has length 0.5; right, they are all shorter.**

The same step in code, digit for digit

```
X = np.array([[1., -2.], [1., 0.], [1., 2.]]) # column 0 = intercept
y = np.array([0., 0., 1.])
w = np.zeros(2)
p = 1 / (1 + np.exp(-(X @ w)))
grad = X.T @ (p - y)
w = w - 0.5 * grad
```

```
p - y          = [ 0.5  0.5 -0.5]
grad = X'(p - y) = [ 0.5 -2. ]
one step, eta = 0.5 : w <- w - eta * grad = [-0.25  1. ]
new p = [0.095349 0.437823 0.851953]
log-likelihood -2.079442 -> -0.836370 (improved by +1.243071)
numerical gradient of the NLL at w = 0: [ 0.5 -2. ]
analytic X'(p - y) : [ 0.5 -2. ]
```

The same step in code, digit for digit

```
X = np.array([[1., -2.], [1., 0.], [1., 2.]]) # column 0 = intercept
y = np.array([0., 0., 1.])
w = np.zeros(2)
p = 1 / (1 + np.exp(-(X @ w)))
grad = X.T @ (p - y)
w = w - 0.5 * grad
```

```
p - y          = [ 0.5  0.5 -0.5]
grad = X'(p - y) = [ 0.5 -2. ]
one step, eta = 0.5 : w <- w - eta * grad = [-0.25  1. ]
new p = [0.095349 0.437823 0.851953]
log-likelihood -2.079442 -> -0.836370 (improved by +1.243071)
numerical gradient of the NLL at w = 0: [ 0.5 -2. ]
analytic X'(p - y) : [ 0.5 -2. ]
```

The last two lines are the point of the slide: a **finite-difference** gradient of the loss agrees with the formula we derived. **If you ever doubt a gradient you have derived, check it this way in three lines.**

Predict first #2

Least squares had a formula. Set the gradient to zero and solve: $X^T X w = X^T y$, one linear system, done.

For logistic regression the same move gives $X^T (\sigma(Xw) - y) = 0$.

Can you solve that for w ?

Predict first #2

Least squares had a formula. Set the gradient to zero and solve: $X^T X w = X^T y$, one linear system, done.

For logistic regression the same move gives $X^T (\sigma(Xw) - y) = 0$.

Can you solve that for w ?

```
least squares: beta = (X'X)^-1 X'y = [-1.5  2.2] -- solved, exactly, in one line
residual gradient X'(Xb - y) = [-0.  0.]
```

```
logistic: the same move gives X'(sigma(Xw) - y) = 0,
           and sigma() cannot be undone to isolate w.
           there is no formula to invert; the equations are transcendental.
           so we iterate.
```

Predict first #2

Least squares had a formula. Set the gradient to zero and solve: $X^T X w = X^T y$, one linear system, done.

For logistic regression the same move gives $X^T (\sigma(Xw) - y) = 0$.

Can you solve that for w ?

```
least squares: beta = (X'X)^-1 X'y = [-1.5  2.2]  -- solved, exactly, in one line
residual gradient X'(Xb - y) = [-0.  0.]
```

```
logistic: the same move gives  X'(\sigma(Xw) - y) = 0,
           and \sigma() cannot be undone to isolate w.
           there is no formula to invert; the equations are transcendental.
           so we iterate.
```

No. And that is a statement about algebra, not about effort

w sits *inside* an exponential inside a fraction inside a sum. There is no rearrangement that isolates it — the same reason $xe^x = 1$ has no elementary solution. **So we do not solve; we descend.** Which is why Unit III came before Unit V.

But we can promise you will not get stuck

The Hessian of the log-likelihood is $H = -X^T SX$ with $S = \text{diag}(p_i(1 - p_i))$, and every $p_i(1 - p_i) > 0$.

w = [0. 0.]	eigenvalues of H	[-7. -1.5]	all < 0 : True
w = [-0.5 1.]	eigenvalues of H	[-2.2368 -0.6133]	all < 0 : True
w = [2. -3.]	eigenvalues of H	[-0.4823 -0.0235]	all < 0 : True

But we can promise you will not get stuck

The Hessian of the log-likelihood is $H = -X^T S X$ with $S = \text{diag}(p_i(1 - p_i))$, and every $p_i(1 - p_i) > 0$.

w = [0. 0.]	eigenvalues of H	[-7. -1.5]	all < 0 : True
w = [-0.5 1.]	eigenvalues of H	[-2.2368 -0.6133]	all < 0 : True
w = [2. -3.]	eigenvalues of H	[-0.4823 -0.0235]	all < 0 : True

Negative definite everywhere $\Rightarrow$ strictly concave

For any $v \neq 0$, $v^T H v = -\sum_i p_i(1 - p_i)(x_i^T v)^2 < 0$ whenever X has full column rank. **One maximum, no local traps, no random restarts.** Compare a neural network, where none of that is true — this is the whole reason logistic regression is still the default first model.

But we can promise you will not get stuck

The Hessian of the log-likelihood is $H = -X^T S X$ with $S = \text{diag}(p_i(1 - p_i))$, and every $p_i(1 - p_i) > 0$.

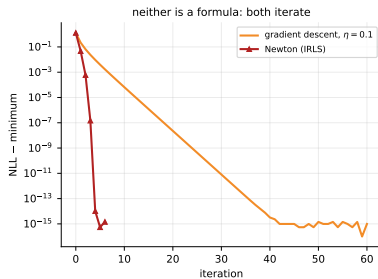
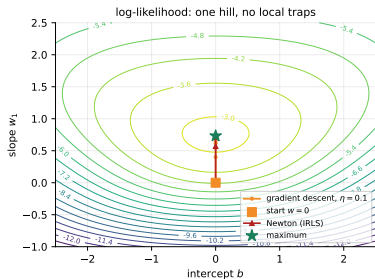
w = [0. 0.]	eigenvalues of H	[-7. -1.5]	all < 0 : True
w = [-0.5 1.]	eigenvalues of H	[-2.2368 -0.6133]	all < 0 : True
w = [2. -3.]	eigenvalues of H	[-0.4823 -0.0235]	all < 0 : True

Negative definite everywhere $\Rightarrow$ strictly concave

For any $v \neq 0$, $v^T H v = -\sum_i p_i(1 - p_i)(x_i^T v)^2 < 0$ whenever X has full column rank. **One maximum, no local traps, no random restarts.** Compare a neural network, where none of that is true — this is the whole reason logistic regression is still the default first model.

Concavity guarantees the optimum is *unique*. It does not guarantee the optimum is *finite* — hold that thought for the last section.

Drawn: one hill, and two ways up it



Six points, two parameters. Left: the contours of ℓ close on a single peak. Orange is gradient descent at $\eta = 0.1$; red is Newton's method, which uses H and arrives in about five steps. Right: the same two runs on a log scale. **Neither is a formula.**

Run it to the end, and compare with the library

iter	b	w1	NLL	grad max
1	+0.000000	+0.400000	3.094797	1.43e+00
2	-0.000000	+0.543158	2.941879	7.28e-01
10	+0.000000	+0.726183	2.876549	2.08e-02
100	-0.000000	+0.732488	2.876484	2.22e-16

sklearn LogisticRegression(penalty=None): b -0.000000 w1 +0.732488
our gradient descent after 20000 steps : b -0.000000 w1 +0.732488
agreement to 11 decimal places

Run it to the end, and compare with the library

iter	b	w1	NLL	grad max
1	+0.000000	+0.400000	3.094797	1.43e+00
2	-0.000000	+0.543158	2.941879	7.28e-01
10	+0.000000	+0.726183	2.876549	2.08e-02
100	-0.000000	+0.732488	2.876484	2.22e-16

sklearn LogisticRegression(penalty=None): b -0.000000 w1 +0.732488
 our gradient descent after 20000 steps : b -0.000000 w1 +0.732488
 agreement to 11 decimal places

Newton step 1:	b +0.000000	w1 +0.571429	grad 4.000e+00
Newton step 3:	b -0.000000	w1 +0.732187	grad 6.322e-02
Newton step 5:	b -0.000000	w1 +0.732488	grad 2.516e-07
Newton step 6:	b -0.000000	w1 +0.732488	grad 1.588e-14

Run it to the end, and compare with the library

iter	b	w1	NLL	grad max
1	+0.000000	+0.400000	3.094797	1.43e+00
2	-0.000000	+0.543158	2.941879	7.28e-01
10	+0.000000	+0.726183	2.876549	2.08e-02
100	-0.000000	+0.732488	2.876484	2.22e-16

sklearn LogisticRegression(penalty=None): b -0.000000 w1 +0.732488
 our gradient descent after 20000 steps : b -0.000000 w1 +0.732488
 agreement to 11 decimal places

Newton step 1:	b +0.000000	w1 +0.571429	grad 4.000e+00
Newton step 3:	b -0.000000	w1 +0.732187	grad 6.322e-02
Newton step 5:	b -0.000000	w1 +0.732488	grad 2.516e-07
Newton step 6:	b -0.000000	w1 +0.732488	grad 1.588e-14

Eleven decimal places between fourteen lines of numpy and scikit-learn. Newton's method — which statisticians call **IRLS** — needs six steps rather than a hundred, because it uses the curvature we just computed.

What a coefficient means

Increase x_j by one unit. Then z increases by β_j , so

$$\frac{\text{new odds}}{\text{old odds}} = \frac{e^{z+\beta_j}}{e^z} = e^{\beta_j}.$$

e^{β_j} is a **multiplicative** change in the **odds**.

It is not a change in probability, and it is not additive

“ $\beta = 0.7$, so the chance goes up by 70%” is wrong twice over. The correct sentence is: **one more unit of x_j multiplies the odds by $e^{0.7} = 2.01$, whatever those odds were.** The change in *probability* depends entirely on where you started.

$\beta_j > 0 \Rightarrow e^{\beta_j} > 1$ (odds up); $\beta_j = 0 \Rightarrow e^{\beta_j} = 1$ (no effect); $\beta_j < 0 \Rightarrow e^{\beta_j} < 1$ (odds down).

By hand: one coefficient, five starting points

Take $\beta = \ln 2 = 0.693147$, so the odds ratio is exactly 2.

start p	start odds	new odds	new p	change in p
0.05	0.0526	0.1053	0.0952	+0.0452
0.20	0.2500	0.5000	0.3333	+0.1333
0.50	1.0000	2.0000	0.6667	+0.1667
0.80	4.0000	8.0000	0.8889	+0.0889
0.95	19.0000	38.0000	0.9744	+0.0244

By hand: one coefficient, five starting points

Take $\beta = \ln 2 = 0.693147$, so the odds ratio is exactly 2.

start p	start odds	new odds	new p	change in p
0.05	0.0526	0.1053	0.0952	+0.0452
0.20	0.2500	0.5000	0.3333	+0.1333
0.50	1.0000	2.0000	0.6667	+0.1667
0.80	4.0000	8.0000	0.8889	+0.0889
0.95	19.0000	38.0000	0.9744	+0.0244

Do the middle row on paper: odds $1 \rightarrow 2$, so $p = 2/(1 + 2) = 0.6667$. **The odds ratio is 2.0 in every row. The change in probability ranges from +0.024 to +0.167 — a factor of 6.8 — from one and the same coefficient.**

By hand: one coefficient, five starting points

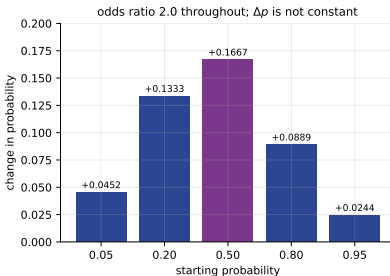
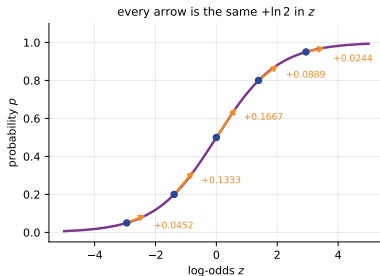
Take $\beta = \ln 2 = 0.693147$, so the odds ratio is exactly 2.

start p	start odds	new odds	new p	change in p
0.05	0.0526	0.1053	0.0952	+0.0452
0.20	0.2500	0.5000	0.3333	+0.1333
0.50	1.0000	2.0000	0.6667	+0.1667
0.80	4.0000	8.0000	0.8889	+0.0889
0.95	19.0000	38.0000	0.9744	+0.0244

Do the middle row on paper: odds $1 \rightarrow 2$, so $p = 2/(1 + 2) = 0.6667$. **The odds ratio is 2.0 in every row. The change in probability ranges from +0.024 to +0.167 — a factor of 6.8 — from one and the same coefficient.**

This is why a paper reports odds ratios and a patient wants probabilities. **They are different questions and you must not mix the answers.**

Drawn: the same arrow, five places on the curve



Every orange arrow is the same horizontal shift of $\ln 2 = 0.693$ in z . The vertical distance it buys depends on how steep the sigmoid is where you are standing — **steepest at $p = 0.5$, nearly flat at the ends.**

Predict first #3

load_breast_cancer, 569 patients. Fit malignancy on mean radius alone, in millimetres. The fitted model is

$$\ln \frac{p}{1-p} = -15.2521 + 1.0340 r.$$

One extra millimetre of radius. By how much does the probability of malignancy rise?

Predict first #3

load_breast_cancer, 569 patients. Fit malignancy on mean radius alone, in millimetres. The fitted model is

$$\ln \frac{p}{1-p} = -15.2521 + 1.0340 r.$$

One extra millimetre of radius. By how much does the probability of malignancy rise?

```
exp(b1) = 2.8124 -- the ODDS multiply by 2.81 per mm, always
radius 10.0 -> 11.0 mm : p 0.0073 -> 0.0203 (+0.0130), odds x 2.8124
radius 14.0 -> 15.0 mm : p 0.3152 -> 0.5642 (+0.2490), odds x 2.8124
radius 17.5 -> 18.5 mm : p 0.9450 -> 0.9797 (+0.0347), odds x 2.8124
radius 20.0 -> 21.0 mm : p 0.9956 -> 0.9984 (+0.0028), odds x 2.8124
```

Predict first #3

load_breast_cancer, 569 patients. Fit malignancy on mean radius alone, in millimetres. The fitted model is

$$\ln \frac{p}{1-p} = -15.2521 + 1.0340 r.$$

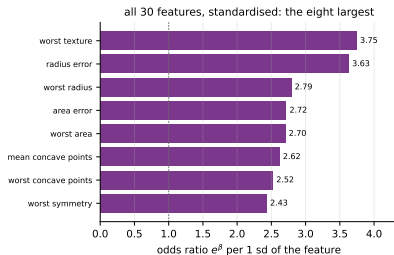
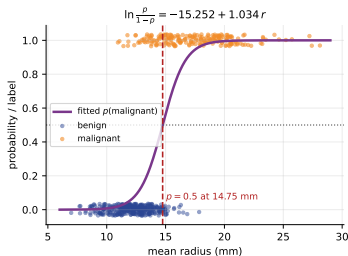
One extra millimetre of radius. By how much does the probability of malignancy rise?

```
exp(b1) = 2.8124 -- the ODDS multiply by 2.81 per mm, always
radius 10.0 -> 11.0 mm : p 0.0073 -> 0.0203 (+0.0130), odds x 2.8124
radius 14.0 -> 15.0 mm : p 0.3152 -> 0.5642 (+0.2490), odds x 2.8124
radius 17.5 -> 18.5 mm : p 0.9450 -> 0.9797 (+0.0347), odds x 2.8124
radius 20.0 -> 21.0 mm : p 0.9956 -> 0.9984 (+0.0028), odds x 2.8124
```

The question has no single answer, and that is the answer

From +0.28 percentage points to +24.9 — a factor of **eighty-nine** — depending on the patient. The odds ratio, 2.8124, is the same in every row. **Divide-by-four rule:** the steepest slope is $\beta/4 = 0.2585$, i.e. about **26** percentage points per mm, and never more.

Drawn: a real fit, and thirty odds ratios



Left: one feature in its own units — the 50% point falls at 14.75 mm, and this single-feature model is already 87.87% accurate. Right: all thirty features standardised, so each bar reads “odds multiplier per one standard deviation”. **Standardise before you compare coefficients, or you are comparing units.**

0.5 is a default, not a law

`predict()` rounds at 0.5. Nothing in the derivation says it must. Held-out set: 171 patients, 64 of them malignant.

thresh	TN	FP	FN	TP	accuracy	precision	recall	F1
0.10	93	14	1	63	0.9123	0.8182	0.9844	0.8936
0.25	95	12	3	61	0.9123	0.8356	0.9531	0.8905
0.40	101	6	3	61	0.9474	0.9104	0.9531	0.9313
0.50	103	4	4	60	0.9532	0.9375	0.9375	0.9375
0.60	106	1	5	59	0.9649	0.9833	0.9219	0.9516
0.75	107	0	6	58	0.9649	1.0000	0.9062	0.9508
0.90	107	0	8	56	0.9532	1.0000	0.8750	0.9333

ROC AUC 0.9917 -- unchanged by the threshold, because it averages over all

0.5 is a default, not a law

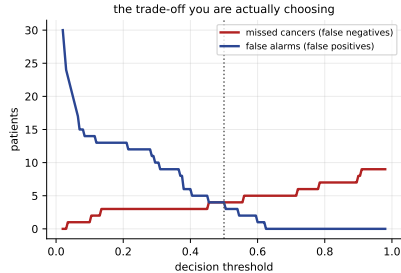
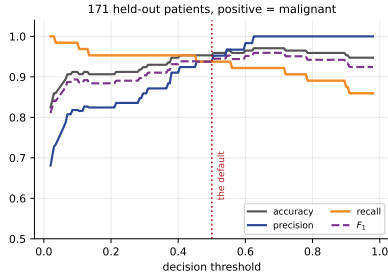
`predict()` rounds at 0.5. Nothing in the derivation says it must. Held-out set: 171 patients, 64 of them malignant.

thresh	TN	FP	FN	TP	accuracy	precision	recall	F1
0.10	93	14	1	63	0.9123	0.8182	0.9844	0.8936
0.25	95	12	3	61	0.9123	0.8356	0.9531	0.8905
0.40	101	6	3	61	0.9474	0.9104	0.9531	0.9313
0.50	103	4	4	60	0.9532	0.9375	0.9375	0.9375
0.60	106	1	5	59	0.9649	0.9833	0.9219	0.9516
0.75	107	0	6	58	0.9649	1.0000	0.9062	0.9508
0.90	107	0	8	56	0.9532	1.0000	0.8750	0.9333
ROC AUC 0.9917 -- unchanged by the threshold, because it averages over all								

The model never changed. Only the rounding rule did

Same 171 probabilities on every row. At 0.10 we miss **one** cancer and raise 14 false alarms; at 0.90 we miss **eight** and raise none. **The best accuracy on this split is 0.9708 at a threshold of 0.62, not at 0.50.** Which row is right is a clinical question, not a statistical one.

Drawn: the trade-off you are actually choosing



Left: four metrics as the threshold sweeps. Precision and recall move in opposite directions by construction. Right: the same picture in patients rather than ratios. **Choose the threshold from the cost of the two mistakes, then report which one you chose.**

From sigmoid to softmax

With K classes, give each class its own linear score $z_k = w_k^\top x + b_k$ and normalise:

$$P(y = k | x) = \frac{e^{z_k}}{\sum_{m=1}^K e^{z_m}} \quad (\text{softmax})$$

Two properties, free of charge

Every term is positive, and they sum to 1 by construction. So the output is a genuine distribution over the K classes, not K unrelated scores.

And $K = 2$ gives back the sigmoid

Put $z_0 = 0$ (only differences matter). Then

$P(y = 1 | x) = \frac{e^{z_1}}{e^{z_1} + 1}$

The loss is the same loss

$$\mathcal{L} = - \sum_i \sum_k \mathbb{I}[y_i = k] \ln P(y_i = k | x_i) —$$

categorical cross-entropy, the multi-class version of L2's loss, and the gradient is again $X^\top (P - Y)$ with Y one-hot.

The alternative: one-vs-rest

Fit K separate binary models, “class k against

Softmax against one-vs-rest, measured

```
soft = make_pipeline(StandardScaler(), LogisticRegression(max_iter=10000))  
ovr = make_pipeline(StandardScaler(),  
                    OneVsRestClassifier(LogisticRegression(max_iter=10000)))
```

```
softmax (multinomial): coef_ shape (3, 4), intercept_ shape (3,)
first three flowers, softmax probabilities:
[0.984703 0.015297 0.         ] sum 1.000000
[0.943549 0.056451 0.         ] sum 1.000000
5-fold accuracy softmax 0.9600 one-vs-rest 0.9267
cross-entropy (log_loss) softmax 0.129534 one-vs-rest 0.280427
with K = 2 the softmax collapses to the sigmoid: True
```

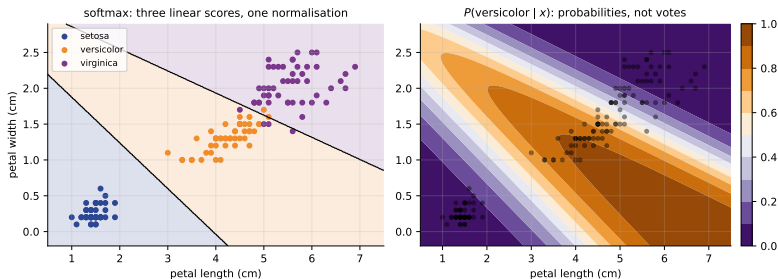
Softmax against one-vs-rest, measured

```
soft = make_pipeline(StandardScaler(), LogisticRegression(max_iter=10000))  
ovr = make_pipeline(StandardScaler(),  
                    OneVsRestClassifier(LogisticRegression(max_iter=10000)))
```

```
softmax (multinomial): coef_ shape (3, 4), intercept_ shape (3,)
first three flowers, softmax probabilities:
  [0.984703 0.015297 0.         ] sum 1.000000
  [0.943549 0.056451 0.         ] sum 1.000000
5-fold accuracy softmax 0.9600 one-vs-rest 0.9267
cross-entropy (log_loss) softmax 0.129534 one-vs-rest 0.280427
with K = 2 the softmax collapses to the sigmoid: True
```

One row of coefficients per class, and the probabilities sum to one exactly. On iris, softmax beats one-vs-rest by 0.033 of accuracy and by more than a factor of two on cross-entropy — because it was trained to produce a *joint* distribution rather than three separate ones.

Drawn: three classes, one normalisation



Left: the boundaries are straight lines, because each pair of classes is separated where $z_j = z_k$, which is linear in x . Right: the probability surface for one class. **Logistic regression is a linear classifier that reports its uncertainty.**

Predict first #4

Six points on a line. Every $x < 0$ is class 0; every $x > 0$ is class 1. **A perfect split exists.** Run unpenalised gradient descent on the negative log-likelihood and keep going.

What does w_1 settle down to?

Predict first #4

Six points on a line. Every $x < 0$ is class 0; every $x > 0$ is class 1. **A perfect split exists.** Run unpenalised gradient descent on the negative log-likelihood and keep going.

What does w_1 settle down to?

steps	w1	NLL	p at x = +1
10	3.3921	6.851e-02	0.967457475172
100	4.8035	1.647e-02	0.991865702976
1000	6.9326	1.952e-03	0.999025489499
10000	9.2132	1.994e-04	0.999900294931
100000	11.5132	1.999e-05	0.999990003302
200000	12.2062	9.998e-06	0.999995000852

Predict first #4

Six points on a line. Every $x < 0$ is class 0; every $x > 0$ is class 1. **A perfect split exists.** Run unpenalised gradient descent on the negative log-likelihood and keep going.

What does w_1 settle down to?

steps	w_1	NLL	p at $x = +1$
10	3.3921	6.851e-02	0.967457475172
100	4.8035	1.647e-02	0.991865702976
1000	6.9326	1.952e-03	0.999025489499
10000	9.2132	1.994e-04	0.999900294931
100000	11.5132	1.999e-05	0.999990003302
200000	12.2062	9.998e-06	0.999995000852

It does not settle. There is no maximum

Every ten-fold increase in steps adds about $2.3 = \ln 10$ to w_1 : the likelihood approaches 1 only as $\|w\| \rightarrow \infty$. The optimum is at infinity, so the MLE **does not exist**. This is *perfect separation*, and it is the one case where concavity does not save you.

What the library does, and what fixes it

```
max_iter  |w1|      NLL      warning
      3    1.8045  3.670e-01  ConvergenceWarning
      5    2.9086  1.125e-01  ConvergenceWarning
     10    6.2700  3.788e-03  ConvergenceWarning
     25   16.6643  1.158e-07  ConvergenceWarning
    100   31.2204  5.607e-14  (none)
the warning text: lbfgs failed to converge after 5 iteration(s) (status=1):
```

What the library does, and what fixes it

max_iter	w1	NLL	warning
3	1.8045	3.670e-01	ConvergenceWarning
5	2.9086	1.125e-01	ConvergenceWarning
10	6.2700	3.788e-03	ConvergenceWarning
25	16.6643	1.158e-07	ConvergenceWarning
100	31.2204	5.607e-14	(none)

the warning text: lbfgs failed to converge after 5 iteration(s) (status=1):

C	w1	b	NLL	p at x = +1
0.01	0.0561	-0.0000	3.83339	0.514017
1.00	1.1044	-0.0000	0.85240	0.751084
100.00	3.9462	+0.0000	0.03905	0.981038

accuracy is 1.0000 at every C -- separation is not a fitting failure

What the library does, and what fixes it

```
max_iter  |w1|      NLL      warning
      3    1.8045  3.670e-01  ConvergenceWarning
      5    2.9086  1.125e-01  ConvergenceWarning
     10    6.2700  3.788e-03  ConvergenceWarning
     25   16.6643  1.158e-07  ConvergenceWarning
    100   31.2204  5.607e-14  (none)
```

the warning text: lbfgs failed to converge after 5 iteration(s) (status=1):

```
      C      |w1|      b      NLL      p at x = +1
    0.01   0.0561  -0.0000   3.83339   0.514017
    1.00   1.1044  -0.0000   0.85240   0.751084
   100.00  3.9462  +0.0000   0.03905   0.981038
```

accuracy is 1.0000 at every C -- separation is not a fitting failure

The fix is regularisation, which is the next-but-one lecture

The L2 penalty adds $\frac{1}{2C} \|w\|^2$, so an infinite w costs infinitely much and **the optimum becomes finite again**. Note that scikit-learn penalises *by default* — which is why you may never have met this bug. **Ridge and lasso: L12.**

And it happens on real data

load_iris, setosa against versicolor, petal length alone.

```
largest setosa 1.9 cm, smallest versicolor 3.0 cm -- separable: True
unpenalised   coefficient      +51.8263 intercept      -127.9249
default L2    coefficient      +2.8999 intercept      -7.8853
the unpenalised coefficient is 18 times larger
both score accuracy 1.0000 and 1.0000 -- the fit is not the problem
unpenalised p at petal length 2.5 cm: 0.8376399347
penalised    p at petal length 2.5 cm: 0.3462536277
```

And it happens on real data

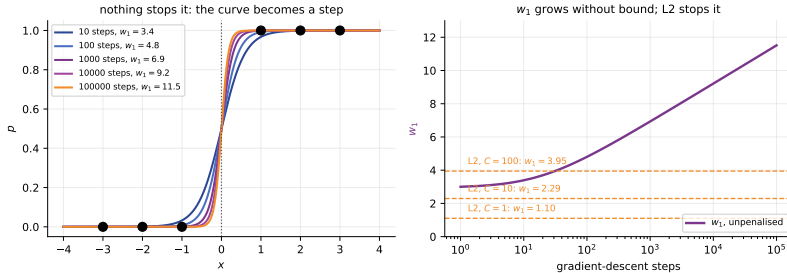
load_iris, setosa against versicolor, petal length alone.

```
largest setosa 1.9 cm, smallest versicolor 3.0 cm -- separable: True
unpenalised   coefficient      +51.8263 intercept      -127.9249
default L2    coefficient      +2.8999 intercept      -7.8853
the unpenalised coefficient is 18 times larger
both score accuracy 1.0000 and 1.0000 -- the fit is not the problem
unpenalised p at petal length 2.5 cm: 0.8376399347
penalised     p at petal length 2.5 cm: 0.3462536277
```

0.84 against 0.35 for the same flower

Both models classify all 100 flowers correctly, so no accuracy figure would ever tell you something was wrong. But a coefficient of **51.8** per centimetre is not a finding, and in the gap between 1.9 and 3.0 cm — where you have **no data** — the two models disagree completely. **Read your coefficients, not just your score.**

Drawn: the runaway, and the brake



Left: the fitted curve at five points along the same run — it is turning into a step function. Right: w_1 against the number of steps, on a log axis, next to where each L2 penalty stops it. **Nothing in the unpenalised problem draws a line.**

Summary — ten lines

1. Least squares on 0/1 labels gave $\hat{y} = -0.1667$ and $+1.1667$ on eight points, and **one distant, correct malignant case moved its boundary from 4.50 to 5.55 mm** and cost it a patient. Logistic regression did not move at all.
2. A probability lives in $(0, 1)$, odds in $(0, \infty)$, log-odds in $(-\infty, \infty)$. **Model the log-odds:**
 $\ln \frac{p}{1-p} = w^T x + b$.
3. Invert that and the sigmoid falls out: $p = 1/(1 + e^{-z})$, with $\sigma(0) = \frac{1}{2}$, $\sigma(-z) = 1 - \sigma(z)$ and $\sigma' = \sigma(1 - \sigma)$, maximum 0.25.
4. Bernoulli likelihood $\prod p_i^{y_i} (1 - p_i)^{1-y_i}$; log-likelihood $\sum [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$, which is **minus the cross-entropy of L2**.
5. $\frac{\partial \ell}{\partial w_j} = \sum_i (y_i - p_i) x_{ij}$, so $\nabla \mathcal{L} = X^T(p - y)$: **prediction minus truth**. The $p(1 - p)$ from the chain rule cancels exactly.
6. Three points, $w = 0$, gave $X^T(p - y) = (0.5, -2.0)$ and $w \leftarrow (-0.25, 1.0)$; the log-likelihood moved $-2.0794 \rightarrow -0.8364$. The code agrees to eleven decimals with `sklearn`.
7. **No closed form:** σ cannot be inverted out of $X^T(\sigma(Xw) - y) = 0$. But $H = -X^T S X$ is negative definite, so ℓ is **strictly concave** — one maximum, no restarts. Newton (IRLS) took 6 steps to gradient descent's 100.
8. e^β multiplies the **odds**, not the probability. $\beta = \ln 2$ moved p by $+0.0244$ from 0.95 and $+0.1667$ from 0.50. On `breast_cancer`, $e^\beta = 2.8124$ per mm of radius, and one extra mm was worth between $+0.28$ and $+24.9$ percentage points.
9. The threshold is **yours**. At 0.10: one missed cancer, 14 false alarms. At 0.90: eight missed, none. Best accuracy was 0.9708 at 0.62, not at 0.50. AUC 0.9917 throughout.
10. Softmax generalises the sigmoid to K classes and beat one-vs-rest 0.9600 to 0.9267 on iris. And when the classes are **perfectly separable the MLE does not exist**: w_1 grew past 12 and kept going, 51.8 on real iris data. **Regularisation is the fix, and that is L12**.

Before the next lecture

Read

notes.pdf — the derivation written out in full, every step, because it is examinable; plus **10 practice questions with worked solutions**.

Do

Questions 2, 4 and 5. Question 4 is one gradient step by hand on two points; question 5 is an odds-ratio sentence that most people get wrong the first time.

Next: multiple linear regression

Many predictors at once, partial and standardised coefficients, and model validation. **The gradient** $X^T(\hat{y} - y)$ **you met today reappears there with** $\hat{y} = Xw$.

And a habit to start now

Whenever you report a logistic coefficient, report e^β next to it and say the sentence out loud: *“one more unit multiplies the odds by ...”* If the sentence sounds wrong, the number is being misread.

Materials: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.; Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.; Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.